

Atelier 2 : Développement et Déploiement d'une API Node.js avec Consommation via React (Array Version)

1. Introduction

Dans le cadre du module Cloud, nous avons réalisé un atelier consistant à :

- Créer une API REST avec Node.js et Express.
- Tester l'API avec Postman.
- Déployer l'API sur Vercel.
- Créer une application React pour consommer cette API.
- Déployer le frontend sur Vercel.

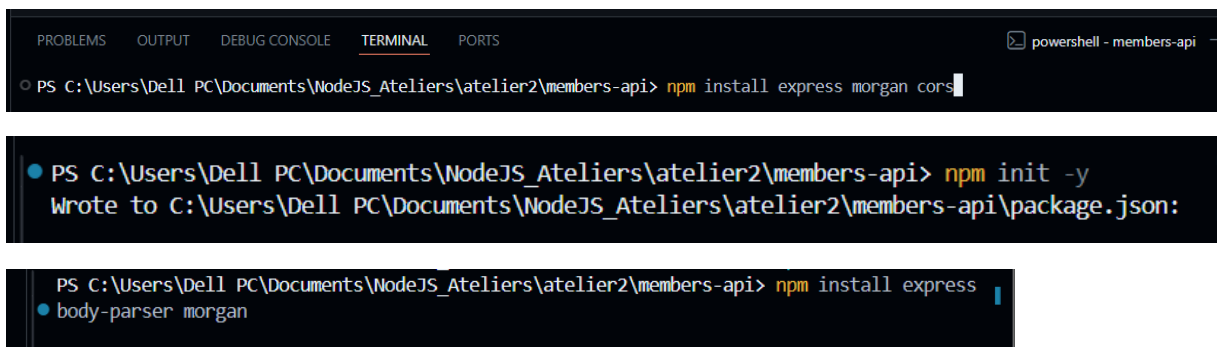
L'objectif principal est de comprendre l'architecture Backend / Frontend séparée utilisée dans les environnements Cloud modernes.

2. Installation des dépendances et test avec Postman

Dans cette étape, nous avons installé les dépendances nécessaires :

- express
- morgan
- cors

Commande utilisée :



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell - members-api
PS C:\Users\De11 PC\Documents\NodeJS_Ateliers\atelier2\members-api> npm install express morgan cors

PS C:\Users\De11 PC\Documents\NodeJS_Ateliers\atelier2\members-api> npm init -y
Wrote to C:\Users\De11 PC\Documents\NodeJS_Ateliers\atelier2\members-api\package.json:

PS C:\Users\De11 PC\Documents\NodeJS_Ateliers\atelier2\members-api> npm install express
body-parser morgan
```

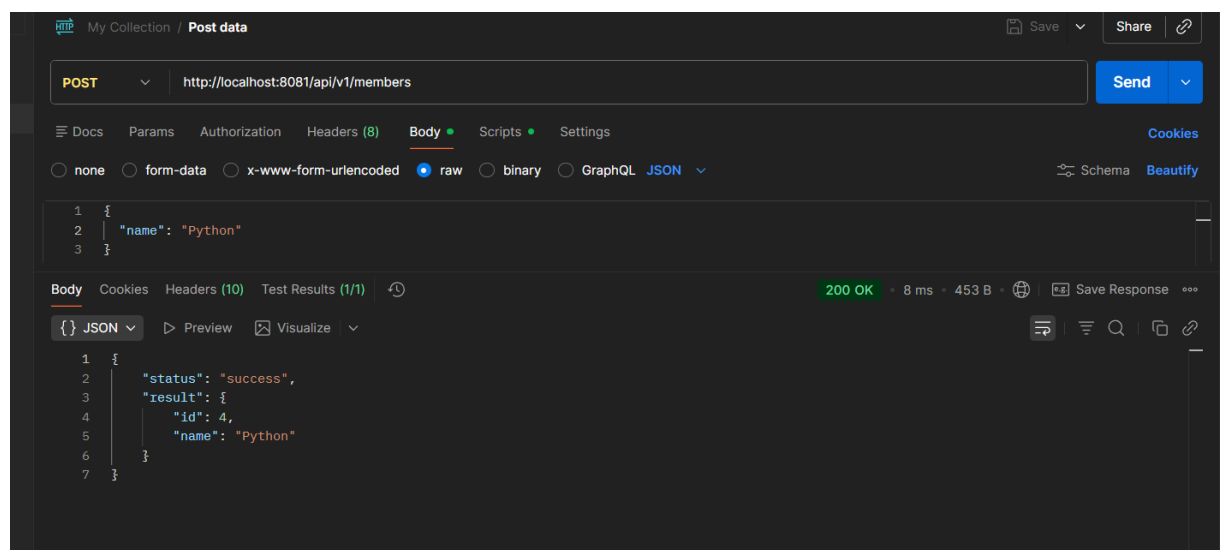
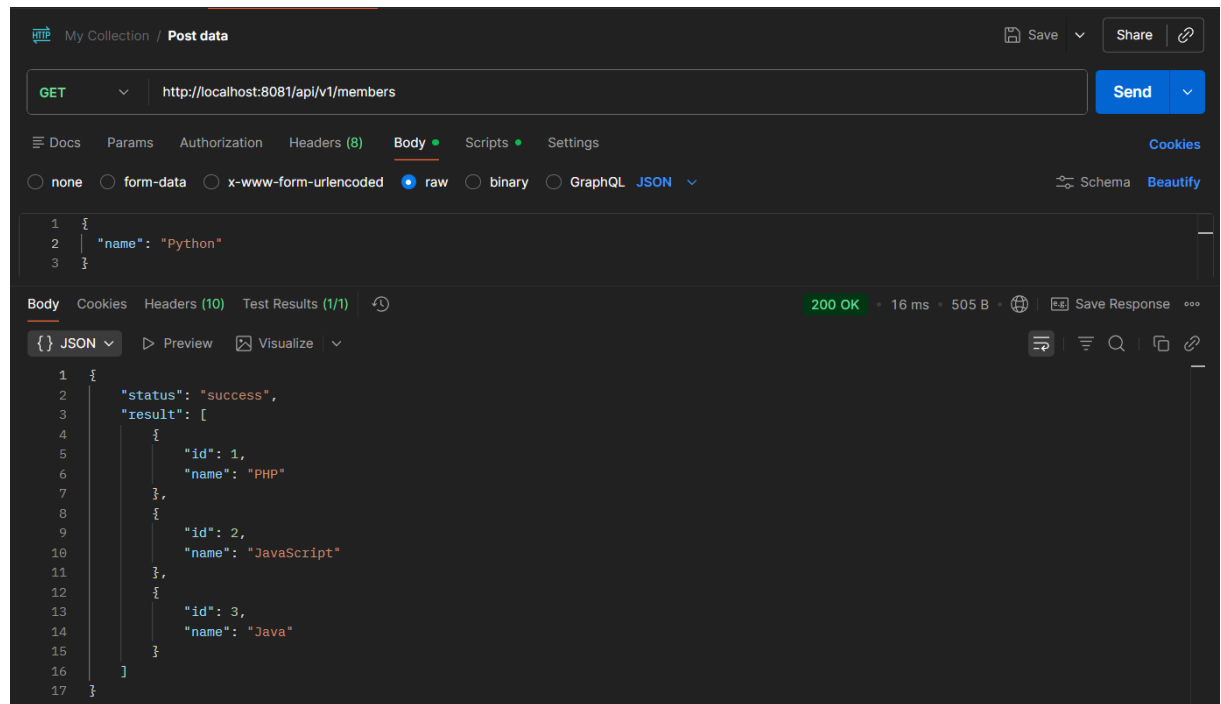
Index.js :

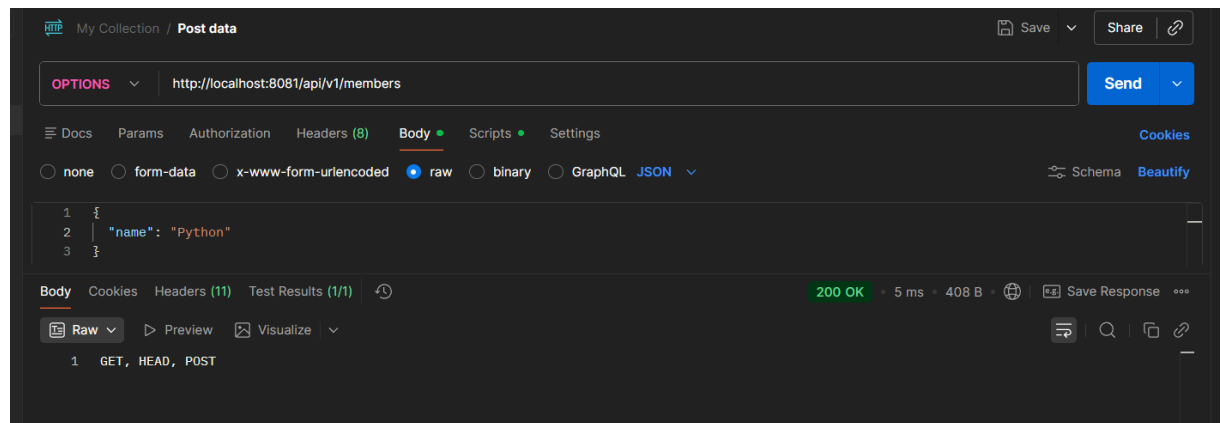
```
JS index.js  X  CSS index.css  main.jsx  App.jsx  CSS App.css
atelier2 > members-api > api > JS index.js > ...
1  const express = require('express')
2  const morgan = require('morgan')
3  const cors = require('cors')
4
5  const config = require('../config.json')
6  const { success, error } = require('../functions')
7
8  const app = express()
9
10 /* ===== MIDDLEWARES ===== */
11
12 // Logger
13 app.use(morgan('dev'))
14
15 // Body parser
16 app.use(express.json())
17 app.use(express.urlencoded({ extended: true }))
18
19 // CORS (IMPORTANT)
20 app.use(cors({
21   origin: [
22     "http://localhost:5173",
23     "https://members-frontend-three.vercel.app"
24   ],
25   methods: ["GET", "POST", "PUT", "DELETE"],
26   allowedHeaders: ["Content-Type"]
27 }))
28
29 // Handle preflight requests
30 app.options('*', cors())
31
32 /* ===== DATA ===== */
33
34 let members = [
35   { id: 1, name: 'Mohamed Said' },
36   { id: 2, name: 'Houdaifa' },
37   { id: 3, name: 'Mostafa' }
```

Ensuite, nous avons testé les routes de l'API avec Postman :

- GET /api/v1/members
- POST /api/v1/members
- PUT /api/v1/members/:id
- DELETE /api/v1/members/:id

Les tests ont confirmé que l'API fonctionne correctement en local.



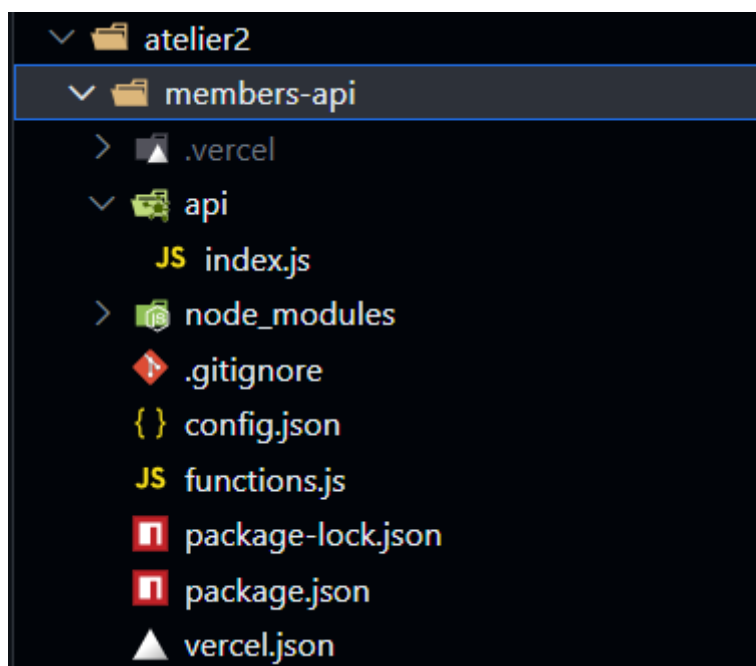


3. Structure du Projet

Le projet backend est structuré comme suit :

- dossier api contenant index.js
- fichier config.json
- fichier functions.js
- fichier package.json
- fichier vercel.json

Cette structure est adaptée au déploiement Serverless sur Vercel.



4. Configuration de l'API

Le fichier config.json contient :

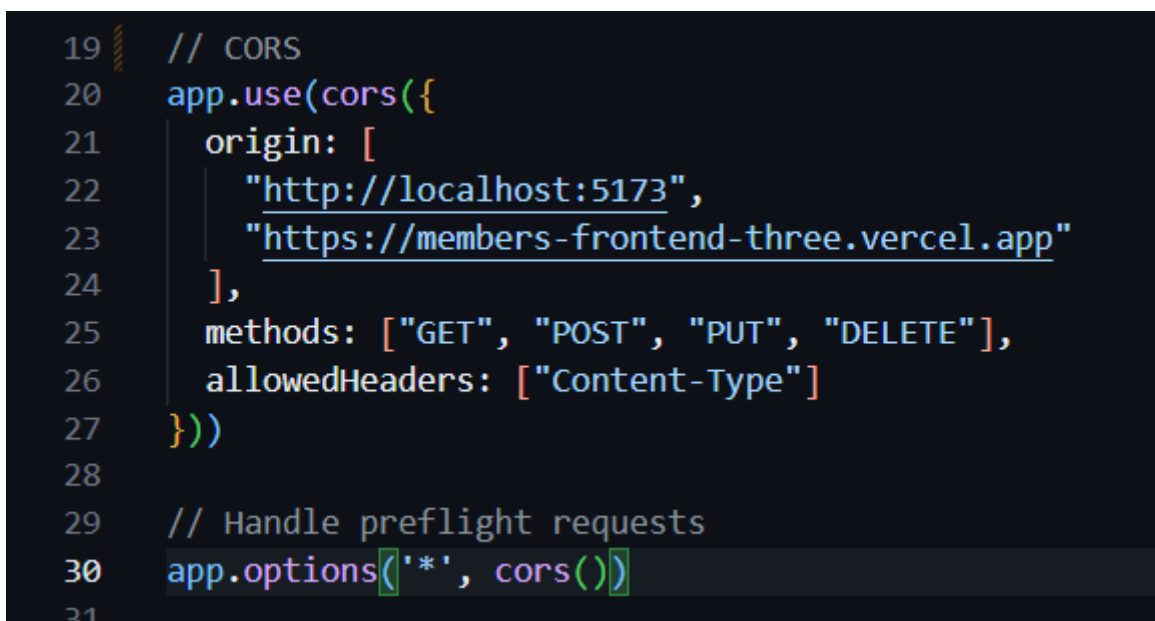


```
Calculator.jsx  .gitignore  vercel.json ...\calculator-api  {} config.json X
atelier2 > members-api > {} config.json > ...
1  {
2    "rootAPI": "/api/v1/",
3    "port": 8081
4  }
```

5. Gestion du CORS

Lors du déploiement, nous avons rencontré un problème CORS.

Pour résoudre ce problème, nous avons utilisé le middleware cors :



```
19 // CORS
20 app.use(cors({
21   origin: [
22     "http://localhost:5173",
23     "https://members-frontend-three.vercel.app"
24   ],
25   methods: ["GET", "POST", "PUT", "DELETE"],
26   allowedHeaders: ["Content-Type"]
27 })))
28
29 // Handle preflight requests
30 app.options('*', cors())
31
```

Cela permet d'autoriser les requêtes provenant du frontend déployé sur un autre domaine.

6. Déploiement sur Vercel

Le déploiement a été réalisé sur la plateforme Vercel.

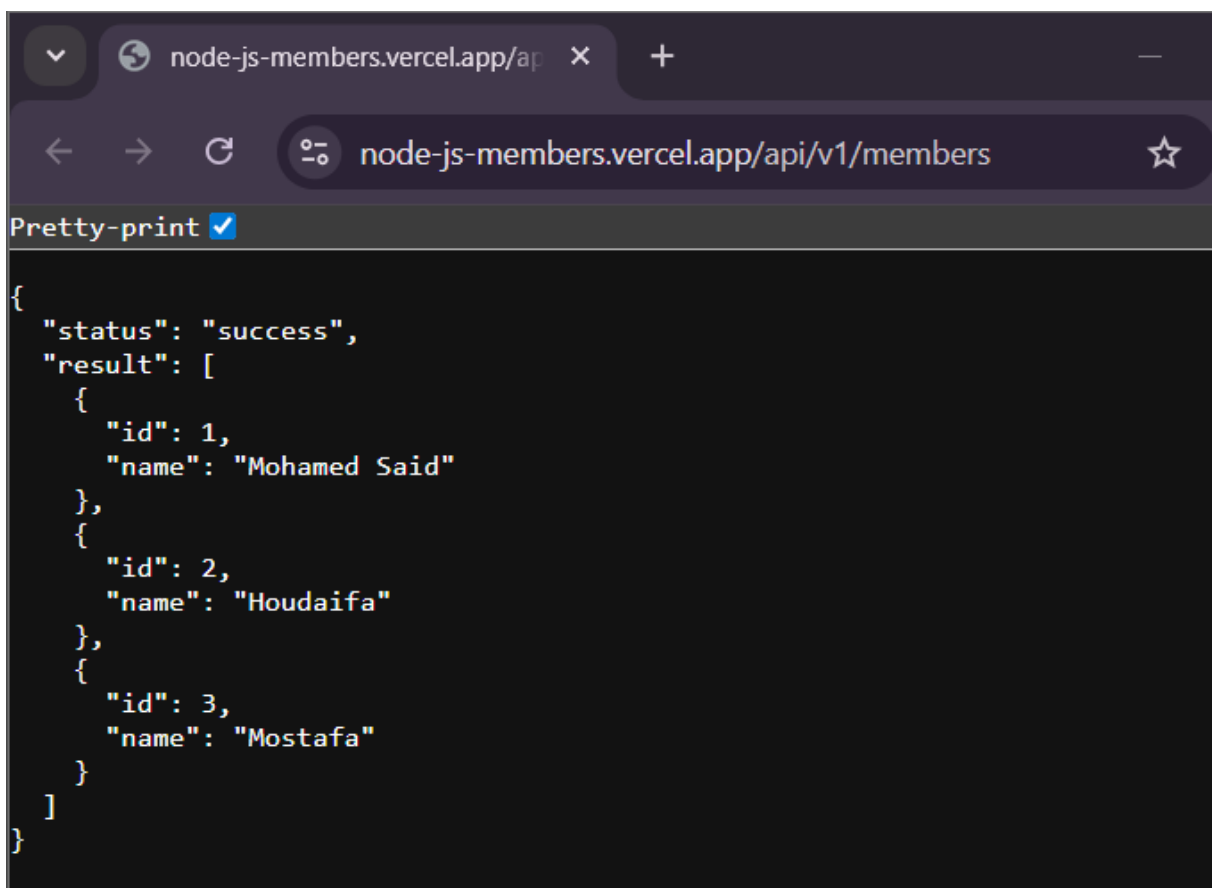
Le fichier vercel.json contient :



```
Calculator.jsx  .gitignore  ▲ vercel.json ..\calculator-api  {} config.json  ▲ vercel.json ..\members-api X
atelier2 > members-api > ▲ vercel.json > ...
1  {
2    "version": 2,
3    "builds": [{ "src": "api/index.js", "use": "@vercel/node" }],
4    "routes": [{ "src": "/(.*)", "dest": "api/index.js" }]
5  }
6  |
```

Après le déploiement, l'API est accessible via :

<https://node-js-members.vercel.app/api/v1/members>



```
node-js-members.vercel.app/api/v1/members
Pretty-print [checked]
{
  "status": "success",
  "result": [
    {
      "id": 1,
      "name": "Mohamed Said"
    },
    {
      "id": 2,
      "name": "Houdaifa"
    },
    {
      "id": 3,
      "name": "Mostafa"
    }
  ]
}
```

7. Création du Frontend React

Une application React a été créée pour consommer l'API.

Axios a été utilisé pour effectuer les requêtes HTTP :

```
App.jsx × App.css Calculator.jsx .gitignore vercel.json ...calculator-api {
atelier2 > members-client > src > App.jsx > App > useEffect() callback
1  import { useEffect, useState } from "react"
2  import axios from "axios"
3  import "../App.css" // Import your new styles
4
5  const API_URL = "https://node-js-members.vercel.app/api/v1/members"
6
7  function App() {
8    const [members, setMembers] = useState([])
9    const [name, setName] = useState("")
10   const [editId, setEditId] = useState(null)
11
12   const fetchMembers = () => {
13     axios.get(API_URL).then(res => setMembers(res.data.result || []))
14   }
15
16   useEffect(() => {
17     fetchMembers()
18   }, [])
19
20   const handleSubmit = () => {
21     if (!name.trim()) return
22     const request = editId
23       ? axios.put(`${API_URL}/${editId}`, { name })
24       : axios.post(API_URL, { name })
25
26     request.then(() => {
27       setName("")
28       setEditId(null)
29       fetchMembers()
30     })
31   }
32
33   const deleteMember = (id) => {
34     if(window.confirm("Delete this member?")) {
35       axios.delete(`${API_URL}/${id}`).then(fetchMembers)
36     }
37   }
}
```

Les données récupérées sont affichées dynamiquement dans l'interface utilisateur.

8. Architecture Cloud

L'architecture finale est composée de :

- Backend Node.js déployé sur Vercel
- Frontend React déployé sur Vercel
- Communication via HTTP REST
- Gestion CORS pour autoriser les échanges entre domaines

Cette architecture reflète un modèle professionnel utilisé dans les environnements Cloud modernes.